Chapter 4 – Common Mistakes in One-Dimensional Arrays

1. Assuming the last index equals the array size

Many beginners confuse the declared array size with the maximum index.
If an array has size N, the valid index range is 0 to N−1.

Example:
int arr[5];
Valid indices: 0,1,2,3,4

Important: The compiler never warns you if you access arr[5]. It silently produces undefined behavior.

2. Treating array names as variables

An array name is a constant memory address. It looks like a pointer, but it cannot be incremented, reassigned, or modified.

Illustration:

Expression	Meaning
arr	constant address of first element
p	pointer variable holding an address
p = p + 1	valid
arr = arr + 1	invalid (constant)

This is one of the most fundamental distinctions between arrays and pointers.

3. Believing that sizeof(arr) works everywhere

Inside the same scope where an array is declared:

```
sizeof(arr)
```

gives the total byte size of the array.
But when passed to a function, the array decays to a pointer, and “sizeof(arr)” becomes the size of a pointer, not the array.

- This is a subtle but dangerous behavior, especially when writing generic functions.

4. Expecting arrays to copy with = operator

Arrays must be copied element-by-element.
= does not copy arrays because arrays are not assignable objects in C.

```
int a[5];
int b[5];

a = b;    // ???
```

"incompatible types… value required… array type is not assignable"

Because:

- a is an address constant.
- b is an address constant.
- A new value cannot be assigned to a fixed address.
- That is, a cannot be changed; it only points to it.

So how to copy an array?

1) using loop:

```
for (int i = 0; i < 5; i++)

    a[i] = b[i];
```

2) or using memcpy():

```
memcpy(a, b, 5 * sizeof(int));
```

5. Forgetting that strings need space for the null terminator

If a string literal has N characters, the array must have space for **N+1**.

Typical mistake:

```
char s[3] = "abc";
```

This overflows because "abc" actually occupies 4 bytes.

6. Mixing up addresses of array vs. address of an element

arr and &arr[0] often produce the same numeric address, but their types are different.

Expression	Type
arr	pointer to first element (int *)
&arr	pointer to whole array (int (*)[N])

This difference affects pointer arithmetic:
&arr + 1 jumps by **N elements**, while
arr + 1 jumps by **1 element**.

7. Misunderstanding pointer reinitialization vs. array modification

Changing a pointer does not change the contents of the array it used to refer to.

Scenario:
A programmer wants to update the array via the pointer:

```
char text[20] = "old text";
char *p = text;
p = "new text";   // only the pointer moves
```

text stays as "old text".

To modify the array, you must assign characters or use strcpy().

8. Using arrays without initialization

An uninitialized array contains **indeterminate values** (garbage).

Important:
Beginners often assume default zero-initialization, which is wrong for local variables.

Safe pattern:
int a[10] = {0};
This clears the entire array automatically.

9. Passing array size incorrectly to functions

A function receiving an array parameter has no way to know how many elements it has.

You *must* pass the size explicitly:

```
void f(int arr[], long size);
```

Common beginner bug:
Assuming sizeof(arr) still works inside the function.

10. Confusing array indexing with pointer arithmetic

Both work similarly, but their mental models differ.
Remember:

```
arr[i] == *(arr + i)
```

But pointer arithmetic depends on the *type size*.
For example, incrementing a pointer to double jumps 8 bytes. This becomes important when switching between pointer-based and index-based code.

11. Thinking arrays can be returned safely from functions

Local arrays inside a function live on the **stack** and disappear after return.

Returning them leads to dangling pointers.

Safer alternatives:

- return a pointer to a static array
- allocate with malloc()
- pass an output array to the function

12. Initializing arrays incorrectly with too many elements

If you put more initializer values than the array size, the compiler gives an error:

```
int a[3] = {1, 2, 3, 4};   // too many elements
```

But if fewer values are provided:

```
int a[5] = {1, 2};
```

- the remaining elements are automatically filled with zeros.